



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

Criação de Imagens

Professor: Henrique Delegrego

Criação de Imagens



Dockerfile

- Arquivo de texto que contém uma série de instruções para construir uma imagem Docker
- O Docker lê o Dockerfile e executa suas instruções passo a passo para criar uma imagem
- Cada instrução cria uma nova camada na imagem, que é armazenada em cache para otimizar futuras compilações
- Uma vez que a imagem é construída, ela pode ser executada como um container

Por que criar um Dockerfile?

- Sem o Docker, seu programa pode funcionar na sua máquina, mas falhar em outros lugares devido a diferenças no sistema operacional, dependências ou configurações
- Com o Docker, você define exatamente qual versão do sistema operacional, quais bibliotecas e dependências seu aplicativo precisa, assegurando que sua aplicação funcione de forma idêntica em todos os lugares

Criação de Imagens

Estrutura

- Um Dockerfile consiste em uma série de pares instrução-argumento
- As instruções não diferenciam maiúsculas de minúsculas, mas são convencionalmente escritas em letras maiúsculas para melhorar a legibilidade
- Comentários começam com **#** e instruções com múltiplas linhas podem usar uma barra invertida \ para continuação

Criação de Imagens

Instruções: FROM

- Define a imagem base a partir da qual uma nova imagem será construída
- Atua como o ponto de partida, estabelecendo o ambiente inicial para todas as instruções subsequentes
- Uma imagem base é uma imagem pré-existente que contém um sistema operacional, bibliotecas, ferramentas ou ambientes de execução de aplicações (por exemplo, Python, Node, Ubuntu, etc)
- Todo Dockerfile deve começar com um **FROM**

```
# Exemplos de Imagens Base Docker
# Um Dockerfile é composto por somente uma instrução FROM

# 1. Ubuntu - Imagem base de Linux para uso geral
FROM ubuntu:24.04

# 2. Alpine - Distribuição Linux leve
FROM alpine:3.20

# 3. Python - Ambiente oficial de execução do Python
FROM python:3.12

# 4. Node.js - Ambiente de execução para JavaScript
FROM node:20

# 5. Nginx - Servidor web de alta performance
FROM nginx:1.27

# 6. Redis - Armazenamento de chave-valor na memória
FROM redis:7

# 7. PostgreSQL - Banco de dados relacional
FROM postgres:16

# 8. MySQL - Banco de dados relacional
FROM mysql:8
```

Criação de Imagens

Instruções: COPY

```
# Copia o artefato de build para o container  
COPY target/meu-aplicativo.jar app.jar
```

- Pega arquivos do contexto de compilação (seja o diretório em que o projeto está ou um arquivo de compilação como um .jar) e os copia para o local desejado no sistema de arquivos da imagem

Instruções: ENTRYPOINT

```
# Comandos para rodar uma aplicação Java, por exemplo  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- Define o executável principal que é executado quando um container é iniciado
- Especifica o comando ou aplicação principal que o container deve executar, estabelecendo efetivamente o propósito do container

Agora:

- Vamos criar o Dockerfile

Criação de Imagens

Build da imagem

- Navegue até o diretório do Dockerfile e abra um CLI nele
- Este diretório servirá como o contexto de build, o que significa que o Docker pode acessar todos os arquivos dentro dele durante o processo de construção da imagem
- O comando é: **docker build -t <nome-imagem>:<tag> .**
 - **docker build:** comando para iniciar o processo de construção da imagem
 - **-t <nome-imagem>:<tag>:** permite que você etiquete sua imagem com um nome e uma tag. Se você omitir a tag, o Docker atribuirá a tag **latest** por padrão
 - **.**: O ponto único significa que o diretório atual é o contexto de build. O Docker irá procurar o Dockerfile dentro deste diretório

```
docker build -t meu-app:1.0 .
```

Agora:

- Vamos criar uma imagem a partir do Dockerfile

Criação de Imagens

Instruções: EXPOSE

- Documenta quais portas uma aplicação ouve e registra essa informação nos metadados da imagem, mas não publica essas portas na rede do host
- Se você omitir a instrução, o container continuará a ouvir na porta para a qual foi configurada internamente, mas o Docker não registrará essa informação nos metadados da imagem

```
# Expõe a porta 8080  
EXPOSE 8080
```

Instruções: WORKDIR

- Define o diretório de trabalho para qualquer instrução **RUN**, **CMD**, **ENTRYPOINT**, **COPY** e **ADD** que o siga

```
# Define o diretório de trabalho  
WORKDIR /app  
  
# Copia o arquivo jar para o container  
COPY target/meu-aplicativo.jar app.jar
```

Agora:

- Vamos criar uma imagem de uma API REST

Criação de Imagens

Instruções: ENV

- Valores dinâmicos que podem influenciar o comportamento de processos, programas e scripts
- Forma de passar dados de configuração para dentro dos containers sem precisar mencionar ao rodar o container
- Essas variáveis estão descritas na documentação do repositório da imagem

```
ENV chave = valor
```

Agora:

- Vamos criar uma imagem de uma aplicação que depende de um banco de dados
- Fazer a atividade Compreendendo Dockerfile



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC